

Oriëntatie op AI

Les 3: algoritmiek - sorteren

2025-2026 Semester 1

HBO-ICT



Agenda

1. Herhaling binair zoeken
2. Selection sort
3. Insertion sort



Binair zoeken

- Probleem: controleer of `target` bestaat in gesorteerde lijst `lst`
- Algoritme:
 1. Zet `mini = 0` en `maxi = lengte van lst - 1`
 2. Zet `index` op het gemiddelde van `mini` en `maxi`
 3. Als element `e` op positie `index` van lijst `lst` gelijk is aan `target`, retourneer succes – `target` is gevonden!
 4. Als element `e` op positie `index` van lijst `lst` kleiner is dan `target`, dan zet `mini = index + 1`
 5. Als element `e` op positie `index` van lijst `lst` groter is dan `target`, dan zet `maxi = index - 1`
 6. Ga naar stap 2 zolang `mini <= maxi`.

Binair zoeken – oefening

2	7	9	12	13	17	22	25	27	28	32	37	39	42	43	47	52	55	57	58	60
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

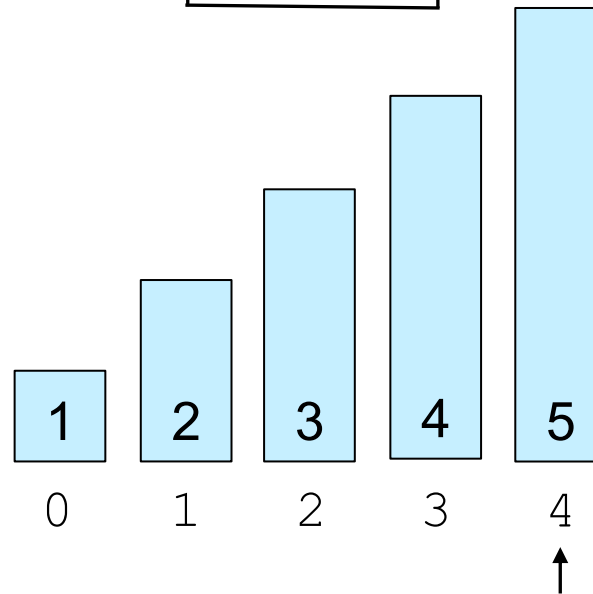
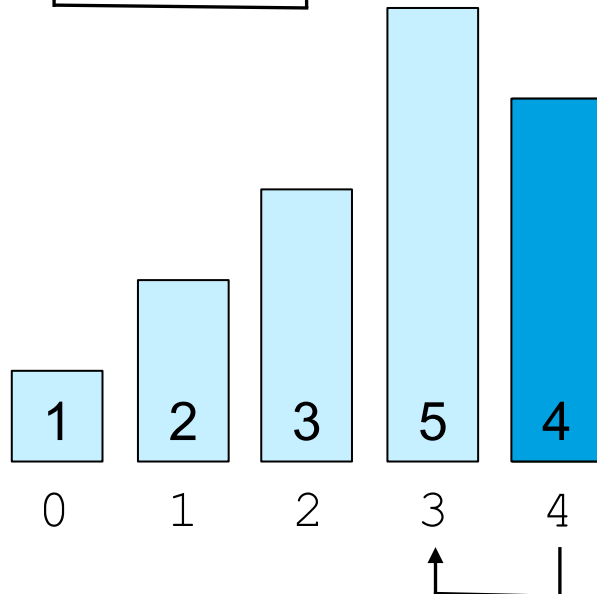
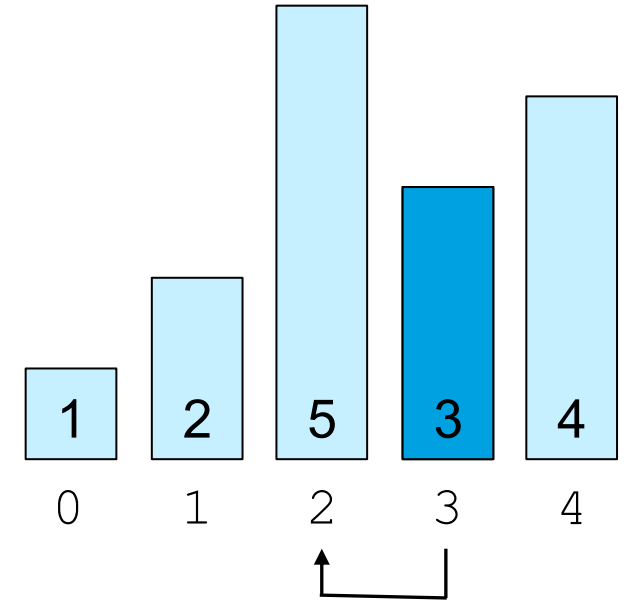
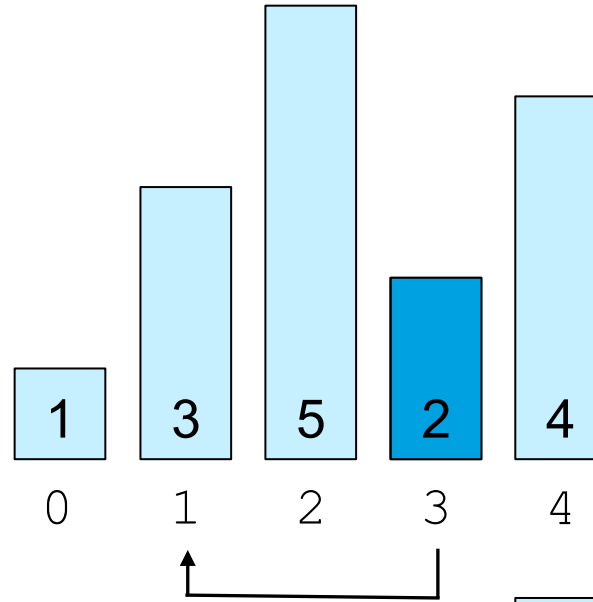
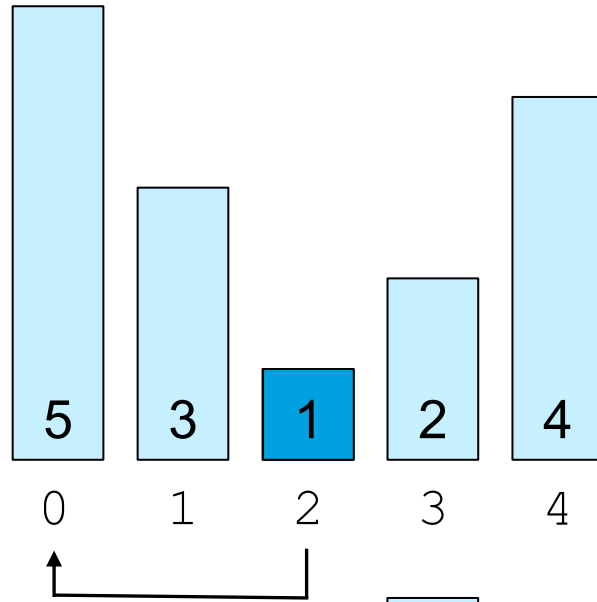
- Gegeven bovenstaande lijst met $n = 21$ elementen
- Hoeveel vergelijkingen heeft het binair zoekalgoritme nodig om een element te vinden *in het slechtste geval*
- Wat was de voorwaarde om binair te kunnen zoeken?
 - Hoe doen we dat?

| Selection sort

Selection sort

- Zoek het kleinste element in de lijst – verwissel deze met het eerste element
- Zoek het op-één-na-kleinste element in de lijst, verwissel deze met het tweede element
- Zoek het op-twee-na-kleinste element in de lijst, verwissel deze met het derde element
- Herhaal deze stappen tot de lijst gesorteerd is

Selection sort



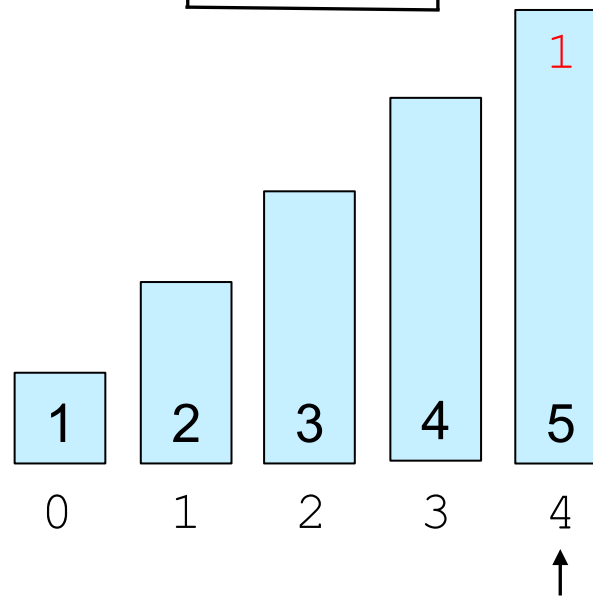
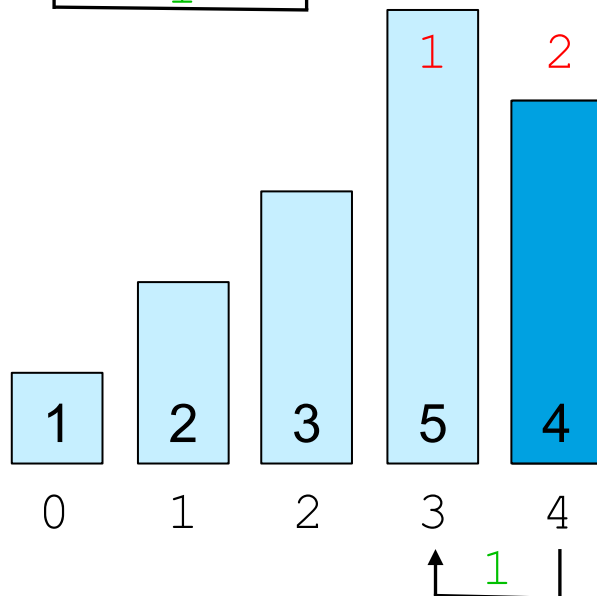
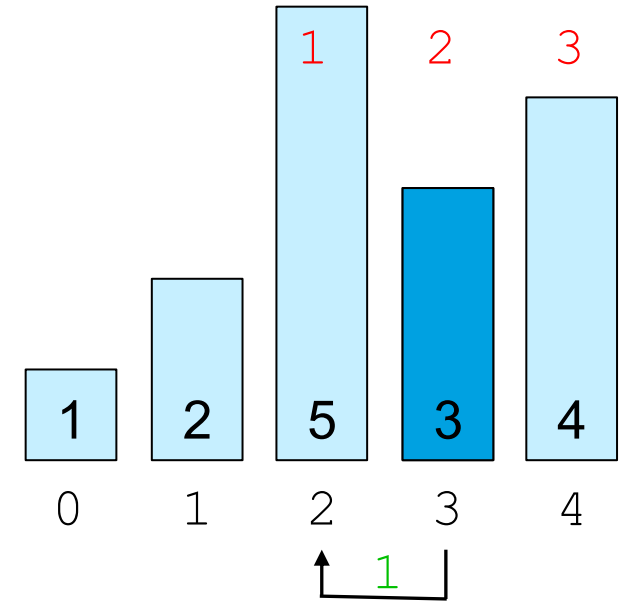
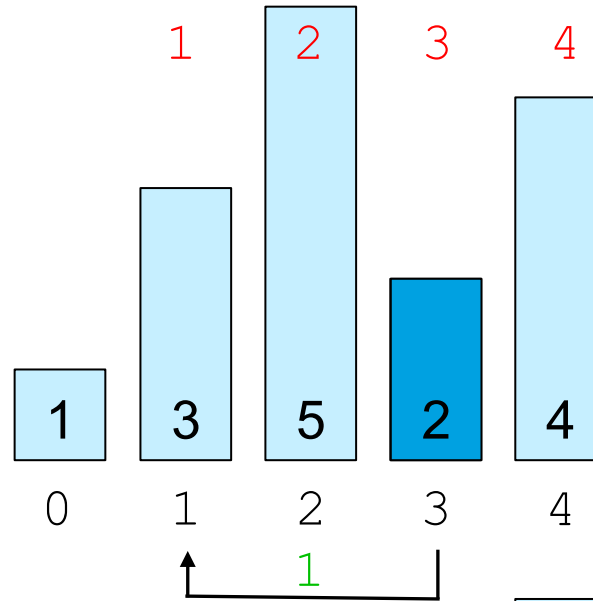
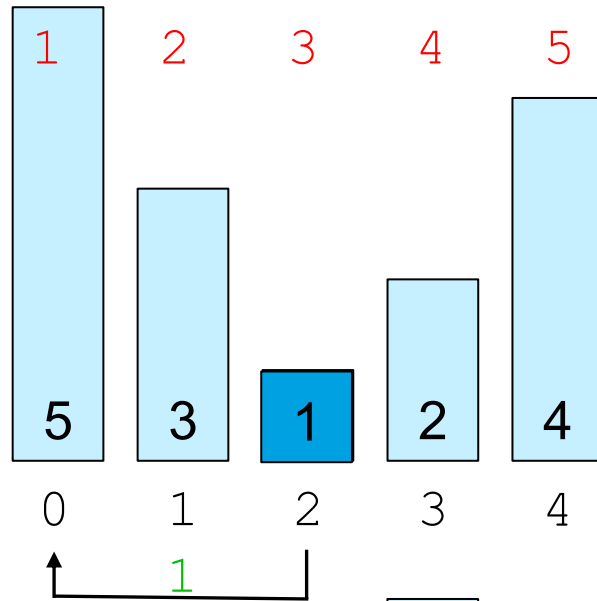
Selection sort – oefening

- Schrijf pseudocode voor het selection sort algoritme

Selection sort – oefening

- Schrijf pseudocode voor het selection sort algoritme
- Implementeer selection sort in Oefening AI3.1: Selection sort (zie Canvas)
 1. Implementeer `swap()`: deze functie verwisselt twee waarden in een lijst
 2. Implementeer `find_index_of_minimum()`: deze functie retourneert de positie van het minimum van een deel van de lijst
 3. Implementeer `selection_sort()` met behulp van bovenstaande functies

Selection sort – analyse



$$5+4+3+2+1 = 15$$

$$(5+1) + (4+2) + 3 =$$

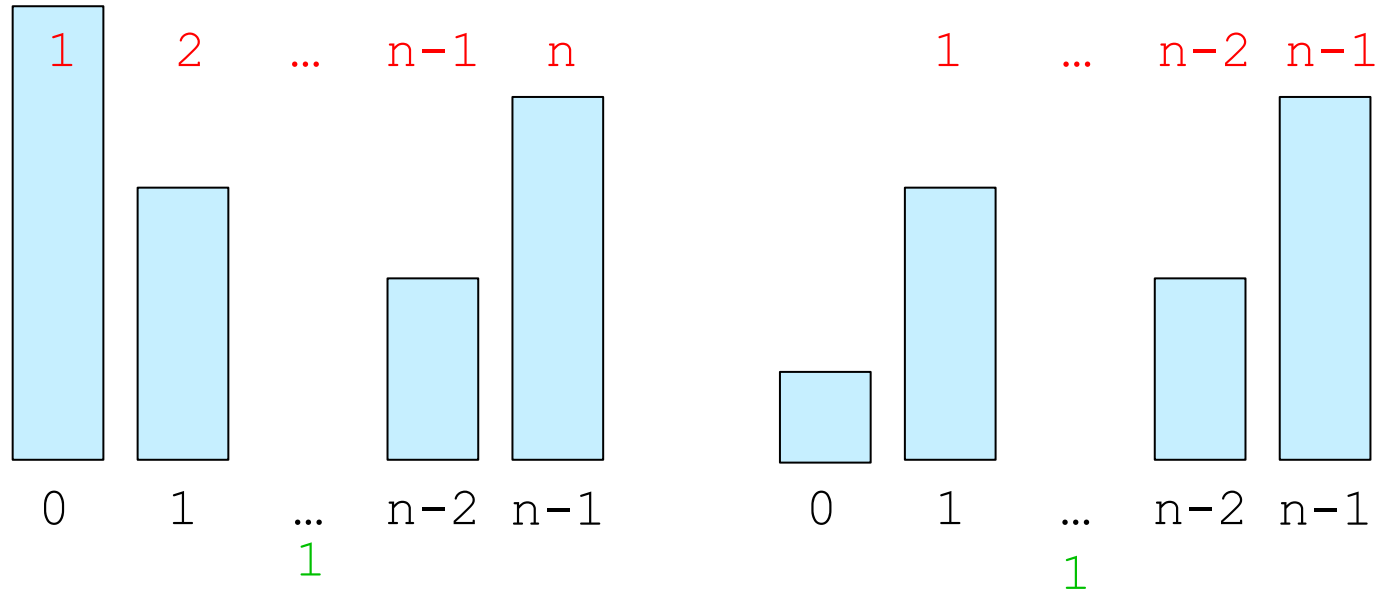
$$6 + 6 + \frac{1}{2} \cdot 6 = 15$$

$$n+1 + n+1 + \frac{1}{2}(n+1)$$

$$1+1+1+1 = 4$$

$$= n-1$$

Selection sort – analyse



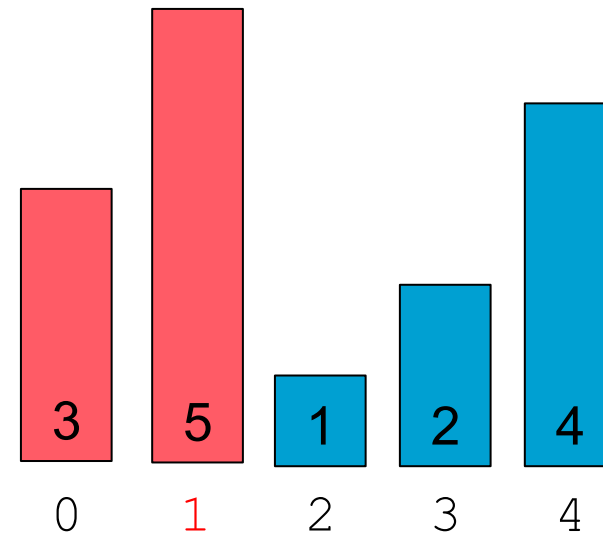
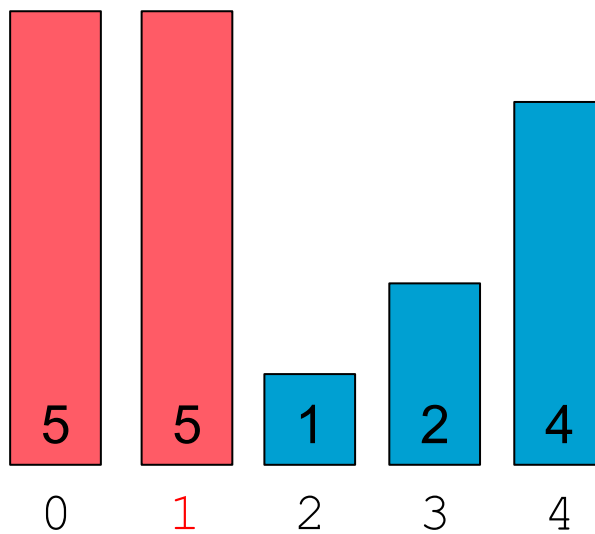
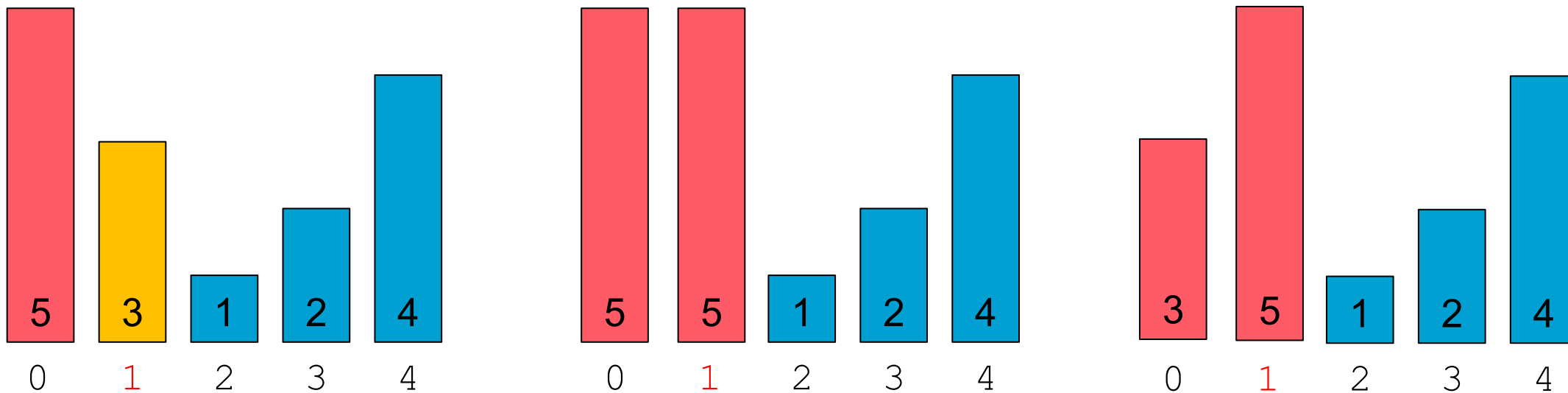
$$\begin{aligned}
 &n + (n-1) + (n-2) + \dots + 3 + 2 + 1 = \\
 &(n+1) + (n-1 + 2) + (n-2 + 3) + \dots = \\
 &(n+1) + (n+1) + (n+1) + \dots = (n+1) * n/2 \\
 &= \frac{1}{2}n^2 + \frac{1}{2}n
 \end{aligned}$$

n-1

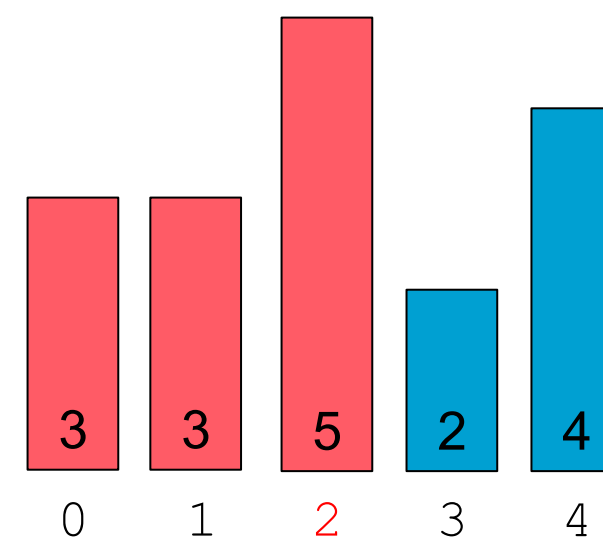
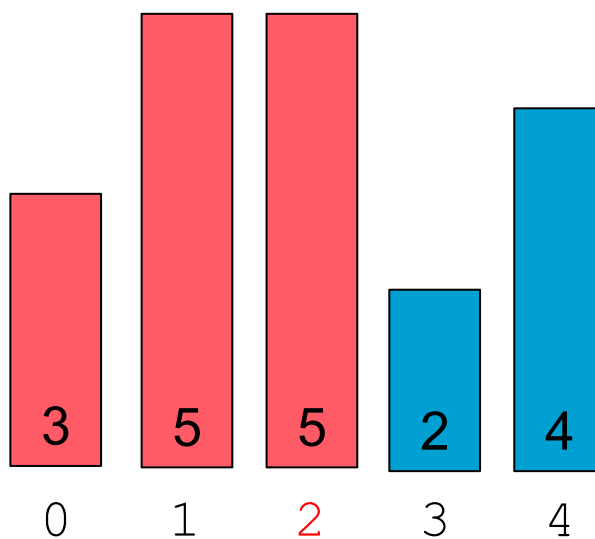
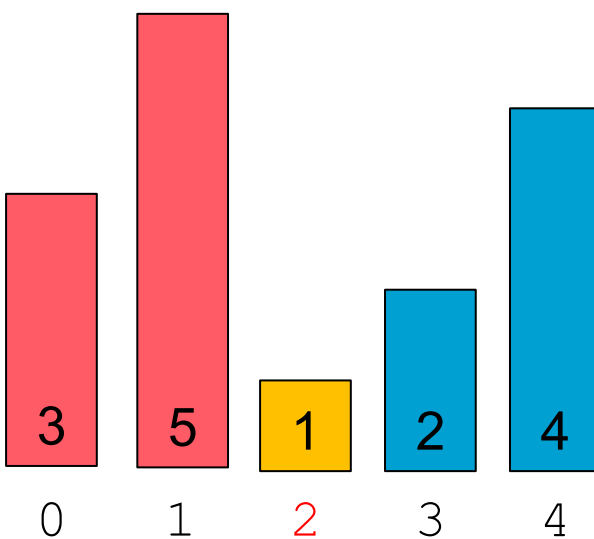
| Insertion sort

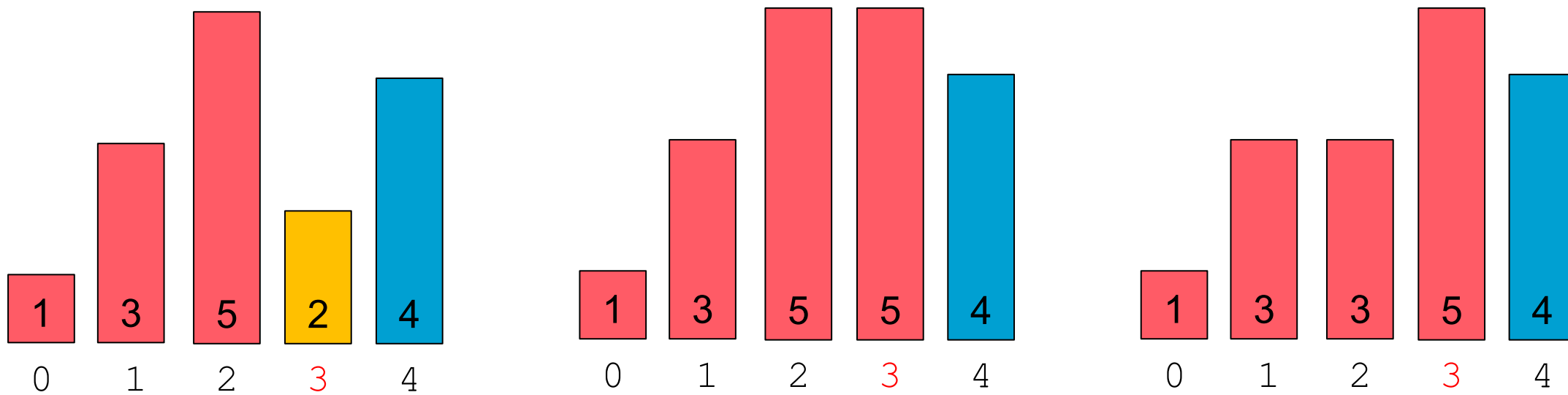
Insertion sort

- Neem element op index 1 en voeg deze op de juiste plek in de reeds gesorteerde deellijst (positie 0)
- Neem element op index 2 en voeg deze op de juiste plek in de reeds gesorteerde deellijst (positie 0 tot en met 1)
- Neem element op index 3 en voeg deze op de juiste plek in de reeds gesorteerde deellijst (positie 0 tot en met 2)
- ...
- Neem element op index n en voeg deze op de juiste plek in de reeds gesorteerde deellijst (positie 0 tot en met $n-1$)



Insertion sort



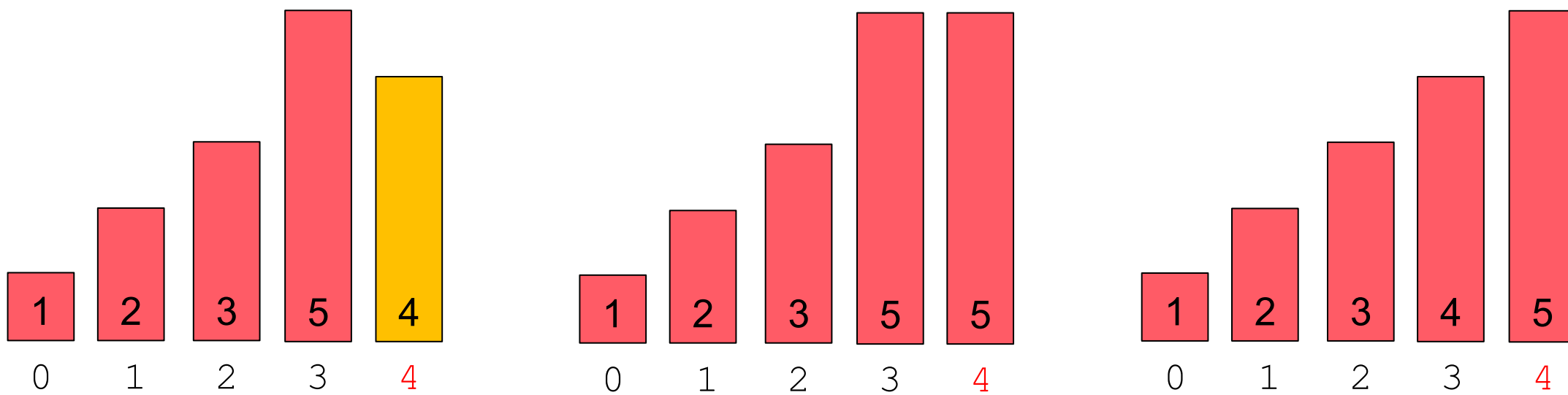


key
2

key
2

key
2

Insertion sort



key
4

key
4

key
4

Insertion sort – oefening

- Bepaal hoeveel stappen (vergelijkingen) insertion sort maakt bij een volledig omgekeerde lijst
 - Bij 3 elementen
 - Bij 4 elementen
 - Bij 5 elementen
 - Bij n elementen
- Wat is de complexiteit van insertion sort in dit geval (worst-case scenario)?

Selection sort – oefening

Implementeer Insertion sort in Oefening AI3.2: Insertion sort (zie Canvas)

- Implementeer `insert()`: deze functie voegt de waarde op de juiste plek van de gesorteerde lijst in

Huiswerk

- Maak de volgende functies af in Python:
 - Selection sort - Oefening AI3.1: Selection sort (zie Canvas)
 - Insertion sort - Oefening AI3.2: Insertion sort (zie Canvas)
- Ga verder met Opdracht AI1: Algoritmiek (zie Canvas)